PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview
2. Background & Business Objective
3. Product Purpose
4. System Functionalities
5. User Stories & Use Cases
6. Features Included (Scope Definition)
7. Features Not Included (Scope Control)
8. Assumptions & Constraints
9. Risks & Questions Throughout Development

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: BioFin Oracle

Version: 1.0

1. Project Overview

- **Problem Statement:** SME orchard owners, comprising 80% of the Malaysian high-value agricultural industry, lack multi-dimensional data integration to counter the "Triple Volatility" crisis (Biological, Meteorological, Market). Reliance on intuition results in 15%-20% wasted budgets on preventative over-fertilization, up to 40% yield collapses from sudden environmental shifts, and massive profit erosion due to market information asymmetry.
- **Target Domain:** Domain 2: AI for Economic Empowerment & Decision Intelligence. Enabling high-level trade-off decisions and rapid operational optimization for agricultural SMEs.
- **Proposed Solution Summary:** BioFin Oracle operates as a "Smart Agri-Fintech Decision Brain." It ingests unstructured market news and structured telemetry to generate explainable, quantifiable action plans via a Next.js dashboard, transitioning farmers from weather-dependent gambles to data-driven operational management.

2. Background & Business Objective

- **Background of the Problem:** Extreme climate unpredictability and international competition (e.g., Thailand, Vietnam) have rendered traditional experience-based farming obsolete. Existing tools offer passive notifications rather than quantifiable damage predictions or operational directives. Delayed decision-making against incoming weather or market dumping consistently results in catastrophic financial losses.
- **Importance of Solving This Issue:** Implementing an intelligence engine that correlates real-time sensor data with market variables mitigates extreme income volatility. Providing actionable directives prevents massive crop loss and resource waste, securing economic stability and enabling modernization for marginalized farmers.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Strategic Fit / Impact:** Aligns directly with Hackathon expectations for decision intelligence. The ILMU-GLM-5.1 reasoning engine is centralized to interpret complex agricultural inputs, outputting quantifiable recommendations that drive measurable economic empowerment through cost reduction and revenue stabilization.

3. Product Purpose

3.1. Main Goal of the System: To function as an AI-powered agricultural risk mitigation and decision intelligence engine that outputs direct, financially optimized action plans.

3.2. Intended Users (Target Audience):

- SME Orchard Owners
- Core Committee Members of Agri-Cooperatives
- Agricultural Data Analysts

4. System Functionalities

4.1. Description: A synchronous Next.js monolith utilizing a serverless API pipeline. The system parses uploaded CSV/JSON agricultural datasets, pre-aggregates the metrics, and injects them into the ILMU-GLM-5.1 reasoning engine to render actionable decision ledgers and digital twin simulations via a React frontend.

4.2. Key Functionalities:

- **Agentic Decision Ledger:** Generates direct operational directives (e.g., altering harvest dates, modifying fertilizer inputs) based on aggregated data.
- **Digital Twin Simulation:** Facilitates dynamic adjustment of inputs (fertilizer, labor) via sliders to calculate impacts on expected net profit using Chart.js visualizations.
- **Heuristic Evaluation Engine:** A pre-processing tier that triggers risk flags based on hardcoded thresholds (e.g., max wind > 24km/h) for baseline analysis.

© UMHackathon 2026

- **Multi-Dimensional Data Ingestion:** Processes up to four distinct datasets: environmental & geospatial data, biological & crop data, farming operations data, and financial & commercial data.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: ILMU-GLM-5.1. Chosen for its capacity to interpret both structured telemetry and unstructured contexts, generating contextualized trade-offs and explainable JSON decision matrices required for the React interface.

4.3.2. Prompting Strategy: Zero-shot data injection. The backend dynamically compiles aggregated arithmetic summaries into a strict prompt template, mandating a deterministic JSON response matching the AnalysisResult TypeScript interface.

4.3.3. Context & Input Handling: Raw file uploads are parsed and statically aggregated within the backend to compress token usage. The system reduces large chronological arrays into specific averages and risk flags before transmitting the payload to the LLM, preventing context window truncation.

4.3.4. Fallback & Failure Behavior: If the ILMU-GLM-5.1 API times out or hallucinates invalid JSON formats, the system automatically routes to a Heuristic Evaluation fallback. This logic uses the hardcoded risk flags to return a pre-written, functional dataset, guaranteeing continuous system operation.

© UMHackathon 2026

5. User Stories & Use Cases

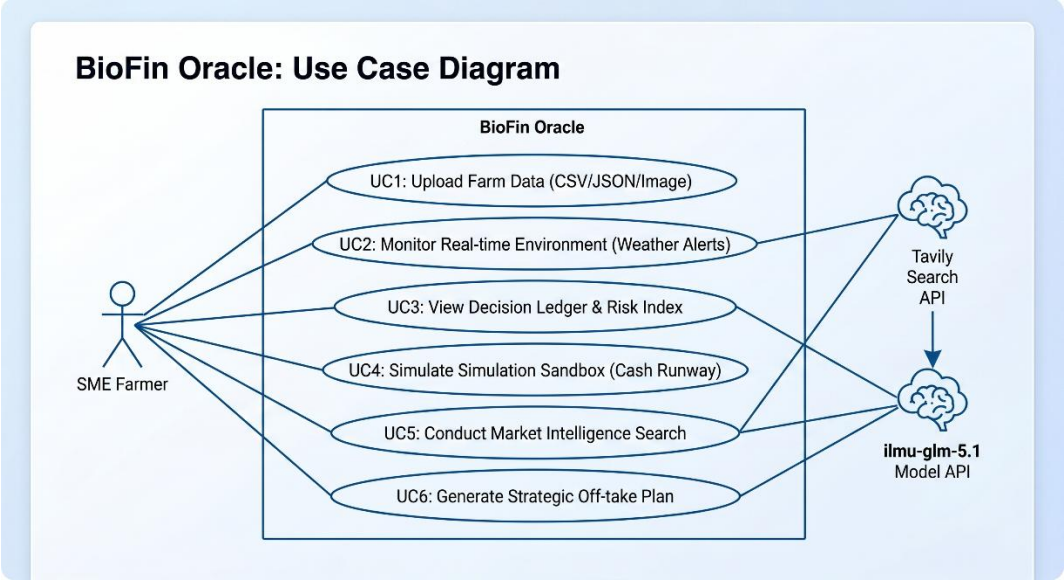
- **User Stories:** "As an SME orchard owner, I want to upload my monthly soil and weather CSVs so that the AI can output a direct action command on whether to harvest now or wait for better market pricing."
- **Use Cases (Main Interactions):**

  Prompt to UI Flow: Users enter the prompt, then Stitch parses the intent, identifies required components, and then renders a customizable 2D Canvas.

  Simulation & Stress Test: User manipulates interface sliders to simulate a prolonged drought; the dashboard recalculates crop viability and profit margins dynamically.

  Decision Execution: System outputs a verified ledger explaining the exact reasoning behind an agricultural recommendation based on market timing and weather forecasts

- **Use Case Diagram:**



© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

- Multi-file drag-and-drop ingestion interface for parsing static CSV and JSON datasets.
- Client-side React dashboard divided into distinct modules: Command Center, Digital Twin Sandbox, Global Stress Test, and Transparency Ledger.
- Interactive Digital Twin Sandbox utilizing sliders to manipulate variables (e.g., Nitrogen Input, Harvest Offset) and visualize profit impacts via dynamic charts.
- Synchronous Next.js backend pipeline executing in-memory mathematical aggregation of uploaded metrics prior to LLM transmission.
- Direct integration with the ILMU-GLM-5.1 API to generate actionable JSON decision ledgers and terminal-style reasoning logs.

7. Features Not Included (Scope Control)

- User authentication protocols, login systems, and multi-tenant concurrent state management
- Persistent database storage; all data processing is strictly ephemeral and destroyed upon session termination.
- Direct hardware integration, live IoT sensor polling, or automated drone imagery acquisition (data is manually uploaded).
- Automated financial execution, banking API integration, or direct supply chain purchasing systems.

8. Assumptions & Constraints

- **LLM Cost Constraint:** Inference costs are aggressively managed by executing a "Lightweight Pre-Aggregation" phase within `route.ts`. The backend statically computes arithmetic averages, chronological trends, and specific anomaly flags (e.g.,

`financial.priceVolatilityPct > 20`) from large raw CSV/JSON arrays before prompt generation. This prevents context window truncation and ensures the LLM processes condensed statistical summaries rather than thousands of raw tokens per session.

- **Technical Constraints:** The system architecture relies on Next.js serverless functions operating without persistent database storage (`Data Storage: None. The system is entirely ephemeral.`). Due to strict Vercel execution timeouts (up to 300 seconds on Vercel Pro), the `fetchMarketIntelligence` pipeline is artificially gated. Tavily API calls are dynamically triggered only when heuristic triggers (e.g., `financial.priceVolatilityPct > 20 or avgPricePerKg < 45`) fire, preventing unnecessary latency.
- **Performance Constraints:** The system is heavily reliant on the structural integrity of the uploaded CSV/JSON files. Missing or malformed columns will cause the `parseCSV` engine to fail. In the event of an ILMU-GLM-5.1 inference failure or schema hallucination, the Next.js API automatically intercepts the error and returns a safe, pre-defined `defaults` object containing hardcoded JSON (`"Graceful AI fallback"`), guaranteeing continuous UI operation at the cost of dynamic personalization.
- **User Input:** The model is completely incapable of performing multi-session trend analysis. Because the system is stateless and lacks a database, all insights, risk factors, and `AnalysisResult` JSON objects generated are strictly isolated to the data payload uploaded during the immediate, current session.

9. Risks & Questions Throughout Development

- **API Latency:** Can the external ILMU-GLM-5.1 endpoint consistently process the aggregated statistical prompts and return the complex `AnalysisResult` JSON payload within the strict Next.js serverless timeout window?

© UMHackathon 2026

- **Schema Hallucination:** Will the GLM model reliably conform to the expected TypeScript interface, or will it inject markdown blocks (e.g., ```json) that corrupt the frontend React hydration? (Addressed via `stripMarkdownJSON` logic).
- **Data Integrity:** How robust is the internal parsing engine (`num` and `avg` functions) against anomalous user inputs or incomplete datasets common in rural agricultural reporting?

© UMHackathon 2026
Email: umhackathon@um.edu.my